

Java 与数据库应用开发 实践报告

题目 银行业务管理系统

院、系 计算机与控制工程学院

专 业 软件工程

班 级 计 243-1

姓 名 王令材

学 号 202458503127

2025 年 11 月 29 日

目录

1 项目描述	1
1.1 系统要解决的问题	1
1.2 本系统实现的功能	1
1.2.1 客户功能	1
1.2.2 管理员功能	1
1.2.3 核心要求	1
1.2.4 安全性要求	1
1.2.5 界面要求	2
2 系统分析	2
2.1 需求分析	2
2.2 用例图	2
2.3 类图设计	3
3 系统设计	4
3.1 开发环境与工具选择	4
3.2 系统总体架构设计	4
3.3 包结构设计	5
3.4 主要类与接口设计	5
3.4.1 BaseDao 类	5
3.4.2 CustomerService 类(客户业务逻辑层)	7
3.4.3 AdminService 类 (管理员业务逻辑层)	7
3.4.4 AdminDao 类(管理员数据访问层)	8
3.4.5 CustomerDao 类 (客户数据访问层)	9
3.4.6 AdminMainFrame 类(管理员核心界面类)	9
3.4.7 CustomerMainFrame 类(客户核心界面类)	10
4 数据库设计	11
4.1 E-R 图设计	11
4.2 物理设计与完整建库语句	12
4.2.1 创建用户表	12

4.2.2 创建管理员表	13
4.2.3 创建交易记录表	13
4.2.4 创建核心视图	14
5 程序设计	14
5.1 转账事务完整伪代码	14
5.2 Swing 线程安全规范	15
5.3 模块结构与调用关系图	16
5.4 核心代码实现	16
5.4.1 转账事务核心代码	16
5.4.2 管理员可编辑表格核心代码	18
6 实现结果与测试	20
6.1 主要界面	20
6.2 客户主要功能实现	21
6.2.1 存款/取款实现	21
6.2.2 转账/查询余额实现	21
6.2.3 修改密码实现	22
6.3 管理员主要功能实现	22
6.3.1 新增交易记录	22
6.3.2 删除选中记录	23
6.3.3 修改选中记录	24
6.3.4 查询实现	24
7 总结	25
7.1 课程设计全过程总结	25
7.2 设计成果的优点和不足	25
7.3 个人学习心得	26
附录：源程序	28

1 项目描述

本系统是一套面向银行柜面场景的完整《银行业务管理系统》,基于 Java 语言与 SQL Server 数据库开发,实现了客户与管理双角色的核心业务流程,为银行个人业务办理提供高效、可靠的数字化解决方案。

1.1 系统要解决的问题

在银行柜面业务中,客户需要进行存款、取款、转账、修改密码、查询余额等操作,管理员需要对所有交易记录进行查询、条件搜索、新增、修改、删除等管理操作。

1.2 本系统实现的功能

1.2.1 客户功能:

涵盖新用户注册、账号登录、存款充值、取款提现、按用户名精准转账(无需记忆复杂用户 ID)、实时余额查询、手机号验证修改密码等全流程服务,操作流程简洁直观,满足客户日常个人金融业务的核心需求。

1.2.2 管理员功能:

支持管理员账号登录、全量交易记录展示、多维度条件组合查询(可通过用户 ID 精准筛选、交易类型分类(存款 / 取款 / 转入 / 转出)、交易日期范围限定),同时提供交易记录的新增、修改、删除等维护操作。

1.2.3 核心要求:

针对存款、取款、转账等涉及资金变动的操作,严格遵循事务一致性原则,即存款业务的“余额增加 + 交易记录生成”、取款业务的“余额扣减 + 交易记录生成”、转账业务的“转出方扣款 + 转入方加款 + 双方交易记录同步插入”等关键步骤,必须同时成功或同时回滚,彻底避免资金账实不符的情况。

1.2.4 安全性要求:

构建多层次安全防护体系,包括登录密码验证、密码加密存储、敏感操作(如转账、改密)的手机号二次验证,同时通过 `PreparedStatement` 预处理 SQL 语句防止

Java 与数据库应用开发实践报告

SQL 注入攻击,全方位保障用户信息与资金安全。

1.2.5 界面要求:

采用 Swing 框架构建可视化界面,设计风格简洁美观,操作流程符合用户使用习惯.

2 系统分析

2.1 需求分析

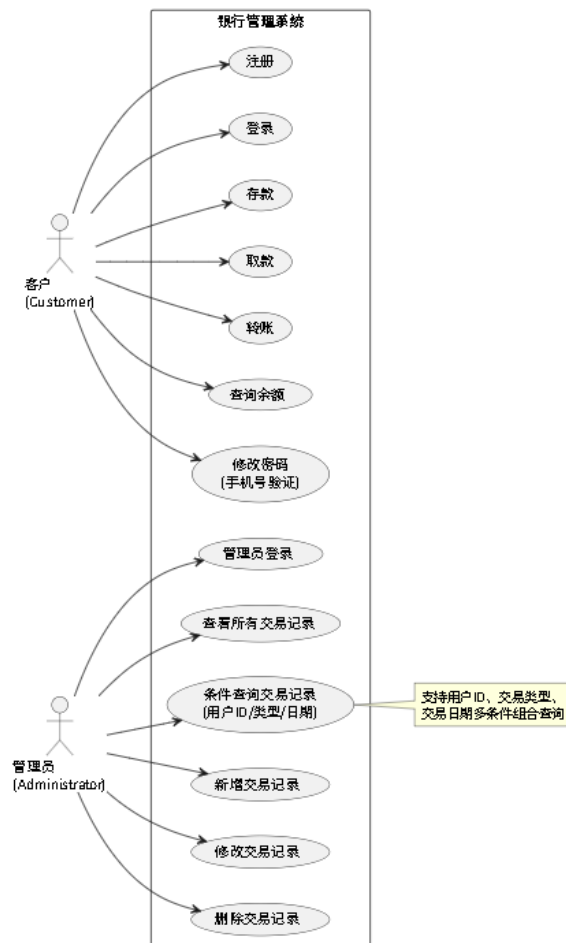
参与者(Actor):客户(Customer) 管理员(Administrator)

主要用例:

客户:注册 → 登录 → 存款 → 取款 → 转账 → 查询余额 → 修改密码

管理员:登录 → 查看所有交易 → 条件查询 → 新增/修改/删除交易记录

2.2 用例图



Java 与数据库应用开发实践报告

图 1 银行系统用例图

- Customer → 登录、注册、存款、取款、转账、查询余额、修改密码
- Administrator → 登录、查询交易记录、条件查询、插入交易、修改交易、删除交易

2.3 类图设计

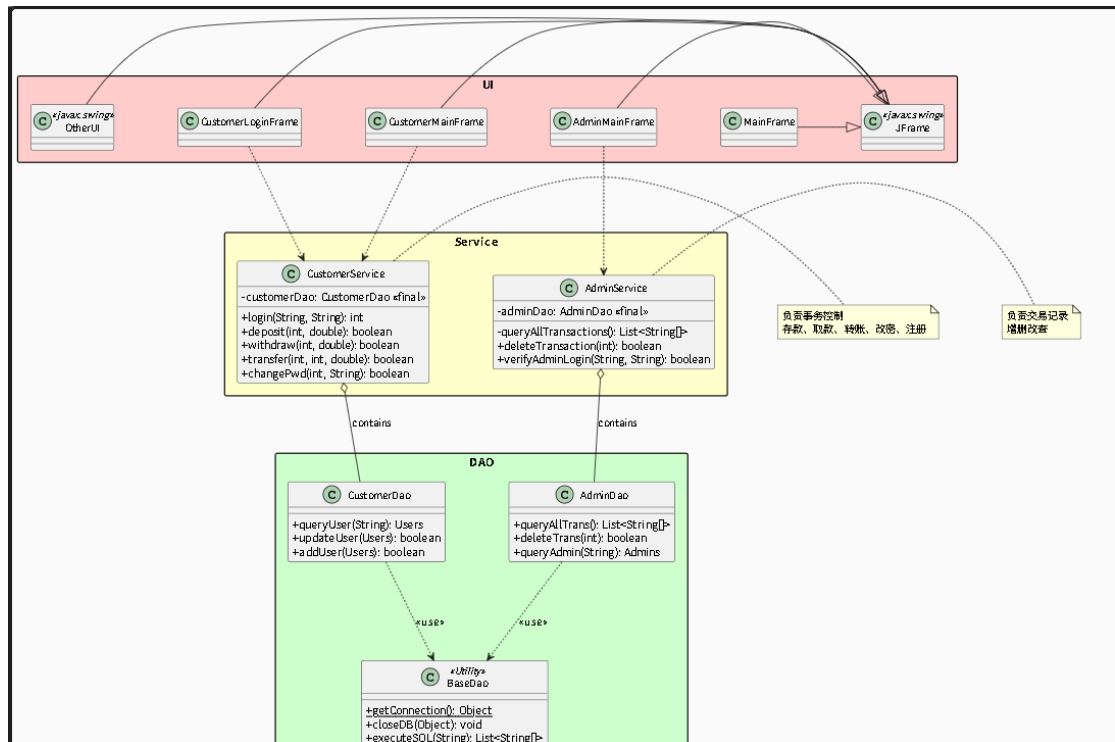


图 2 系统类图

- Users(实体):
CustomerDao ↔ CustomerService ↔ CustomerLoginFrame / CustomerMainFrame
等 UI 类
- Admins(实体):
AdminDao ↔ AdminService ↔ AdminLoginFrame / AdminMainFrame
- Transactions(实体):
TransactionView(视图)
- BaseDao:
工具类,提供静态 getConnection()

Java 与数据库应用开发实践报告

- 继承关系:所有窗口类继承 JFrame
- 依赖关系:所有 UI 类依赖 Service 层

3 系统设计

3.1 开发环境与工具选择

- 开发工具:IntelliJ IDEA 2025.2
- JDK 版本:Java 21
- 数据库:SQL Server Management Studio 21
- JDBC 驱动:mssql-jdbc-13.2.1.jre11.jar
- 建模工具:Visual Studio Code PMDL

3.2 系统总体架构设计

采用经典的三层架构(严格分层,无跨层调用):



附加工具层:

BaseDao(连接池基础)

UIUtils(按钮美化)

ImageBackgroundPanel(背景图)

3.3 包结构设计

```
src
├─ dao
│   ├── AdminDao
│   ├── BaseDao
│   ├── BaseDao.class
│   └── CustomerDao
├─ init
│   └── InstallDatabase.java
├─ main
│   └── Main
├─ service
│   ├── AdminService
│   └── CustomerService
└─ ui
    ├── AdminLoginFrame
    ├── AdminMainFrame
    ├── ChangePwdFrame
    ├── CustomerLoginFrame
    ├── CustomerMainFrame
    ├── DepositFrame
    ├── ImageBackgroundPanel
    ├── MainFrame
    ├── RegisterFrame
    ├── TransferFrame
    ├── UIUtils
    └── WithdrawFrame
```

图 3 根目录下主要文件

3.4 主要类与接口设计

3.4.1 BaseDao 类

BaseDao 是位于 dao 包下的核心数据库工具类,核心目标是统一封装 JDBC 数据库连接逻辑,将连接参数、连接获取逻辑集中管理,避免 CustomerDao、AdminDao 等数据访问类重复编写连接代码,大幅降低代码冗余和后续维护成本,其核心实现与设计细节如下:

私有静态常量: 数据库连接参数 (不可变、全局共享)

```
private static final String URL =
```

```
"jdbc:sqlserver://localhost:1433;databaseName=BankDB;trustServerCertificate=true;;";
```

```
private static final String USER = "sa";
```


Java 与数据库应用开发实践报告

```
private static final String PASSWORD = "123456";
```

核心静态方法：统一获取数据库连接

```
public static Connection getConnection() throws SQLException {  
    return DriverManager.getConnection(URL, USER, PASSWORD);  
}
```

方法定位：`public static` 修饰，允许所有 DAO 类直接通过 `BaseDao.getConnection()` 调用，无需实例化 `BaseDao`；

返回值：`java.sql.Connection` 对象，是 JDBC 操作数据库的核心入口（后续执行 SQL、处理事务均依赖此对象）；

异常处理：声明抛出 `SQLException`（数据库连接异常），不做内部捕获——将异常抛给调用者（如 `CustomerDao`、`AdminDao`），由调用者根据业务场景处理；
内部逻辑：

依赖 `DriverManager`（JDBC 核心类）的 `getConnection` 方法，传入 URL、账号、密码，自动完成“加载 SQL Server 驱动 → 建立 TCP 连接 → 返回连接对象”的全流程；

无需手动管理驱动加载：SQL Server JDBC 驱动（`mssql-jdbc-13.2.1.jre11.jar`）会自动注册到 `DriverManager`，简化代码。

代码复用：`CustomerDao` 的 `login`、`deposit`，`AdminDao` 的 `getAllTransactions`、`insertTransaction` 等方法，均通过 `BaseDao.getConnection()` 获取连接，避免每个方法重复编写 `DriverManager.getConnection(URL, USER, PASSWORD)`；

异常分层处理：连接异常抛给业务 DAO 层，DAO 层可结合业务场景处理（如 `AdminDao` 中捕获异常后弹窗提示“加载交易记录失败”，`CustomerDao` 中提示“登录验证失败”），更贴合用户交互。`CustomerDao` 类(客户数据访问层)

负责客户相关的数据库原子操作,所有方法均依赖 `BaseDao` 获取连接,且涉及资金变动的方法接收外部传入的 `Connection` 对象以支持事务控制,核心方法如下:

```
int login(String username, String password):
```

验证客户登录信息,匹配成功返回用户唯一标识 `user_id`,匹配失败返回 `-1`;

Java 与数据库应用开发实践报告

`boolean deposit(int userId, double amount, Connection conn):`

执行存款操作,接收事务级 `Connection` 对象,完成 “用户余额增加 + 交易记录插入” 的数据库操作,成功返回 `true`,失败返回 `false`;

`boolean withdraw(int userId, double amount, Connection conn):`

执行取款操作,校验用户余额充足性后,完成 “用户余额扣减 + 交易记录插入”,依赖外部事务连接保证操作原子性;

`boolean transfer(int fromUserId, int toUserId, double amount, Connection conn):`

核心转账数据库操作,接收转出方 ID、转入方 ID、转账金额及事务连接,完成 “转出方扣款 + 转入方加款 + 双方交易记录插入” 的原子操作;

辅助方法:`boolean verifyPassword(int userId, String password)`(验证密码)、

`boolean verifyPhone(int userId, String phone)`(验证手机号)、`boolean pdatePassword(int userId, String newPwd)`(修改密码)、`double getBalance(int userId)`(查询余额)、`int getUserIdByUsername(String username)`(通过用户名查询用户 ID)。

3.4.2 CustomerService 类(客户业务逻辑层)

作为客户业务的核心管控层,承接 UI 层的操作请求,封装业务逻辑并统一控制事务,核心特性如下:

事务管控:所有涉及资金变动的操作(存款、取款、转账)均在此层开启手动事务:通过 `conn.setAutoCommit(false)`关闭自动提交,业务执行成功则调用 `conn.commit()`提交事务,执行失败则调用 `conn.rollback()`回滚事务,确保资金操作的原子性;

核心方法:包含 `register(String username, String password, String phone)`方法,完整实现新用户注册流程(校验用户名唯一性→插入用户数据→初始化余额为 0); 同时封装 `deposit()`、`withdraw()`、`transfer()`等方法,调用 `CustomerDao` 的对应数据库操作并增加业务规则校验(如取款金额 $\leq$ 余额、转账金额 > 0 等)。

3.4.3 AdminService 类 (管理员业务逻辑层)

作为管理员业务的核心管控层,承接 UI 层 (`AdminMainFrame`) 的操作请求,封装管理员相关业务逻辑,核心特性如下:

Java 与数据库应用开发实践报告

核心职责：通过实例化 AdminDao 完成所有数据库交互，无额外业务规则校验，仅转发 UI 层请求至 DAO 层；

核心方法：封装 getAllTransactions ()（查询所有交易记录）、searchTransactions ()（多条件查询，含重载方法兼容现有代码）、insertTransaction ()/updateTransaction ()/deleteTransaction ()（交易记录增删改）、verifyAdmin ()/login ()（管理员登录验证），直接调用 AdminDao 对应方法完成操作。

3.4.4 AdminDao 类(管理员数据访问层)

作为管理员模块的核心数据访问类,基于 BaseDao 统一获取数据库连接,专注于交易记录的全生命周期管理及管理员登录验证,核心方法如下:

List<String[]> getAllTransactions():

查询所有交易记录(含存款 / 取款 / 转账),从 TransactionView 读取数据并按交易时间降序返回(字段含交易 ID、用户信息、类型、金额、时间);

List<String[]> queryTransactionsByCondition(String userId, String type, String transDate):

多条件查询交易记录,支持按发起用户 ID、交易类型、交易日期筛选,动态拼接 SQL;

boolean insertTransaction(int userId, String type, double amount, Integer targetUserId):

新增交易记录,自动填充当前时间,targetUserId 可为 null(适配存款 / 取款场景),成功返回 true;

boolean updateTransaction(int transId, String newType, double newAmount, Integer targetUserId):

按交易 ID 修改记录的类型、金额、目标用户 ID,targetUserId 可为 null,成功返回 true;

boolean deleteTransaction(int transId):

按交易 ID 删除指定记录,成功返回 true;

boolean verifyAdmin(String username, String password):验证管理员账号密码,匹

Java 与数据库应用开发实践报告

配返回 true,异常时弹窗提示并返回 false。

3.4.5 CustomerDao 类（客户数据访问层）

作为客户模块的核心数据访问类，基于 BaseDao 统一获取数据库连接，专注于客户登录验证、资金操作（存款 / 取款 / 转账）及账户信息管理，核心方法如下：

`int login(String username, String password)`: 验证客户登录信息，匹配返回用户 ID，失败返回 -1；

`boolean verifyPassword(int userId, String password)`: 校验客户密码，用于存款 / 取款等敏感操作验证，匹配返回 true；

`boolean deposit(int userId, double amount, Connection conn)`: 接收事务连接执行存款操作，完成余额更新 + 存款交易记录插入，成功返回 true；

`boolean withdraw(int userId, double amount, Connection conn)`: 接收事务连接执行取款操作，先校验余额再更新余额 + 插入取款记录，成功返回 true；

`boolean transfer(int fromUserId, int toUserId, double amount, Connection conn)`: 接收事务连接执行转账操作，校验余额 / 转入用户后，完成双方余额更新 + 插入转出 / 转入记录，成功返回 true；

`boolean verifyPhone(int userId, String phone)`: 验证客户手机号，用于修改密码前校验，匹配返回 true；

`boolean updatePassword(int userId, String newPassword)`: 修改客户密码，成功返回 true；

`int getUserIdByUsername(String username)`: 通过用户名查询用户 ID，转账时匹配转入方，存在返回 ID，失败返回 -1；

`double getBalance(int userId)`: 根据客户 ID 查询账户余额，成功返回余额，异常 / 用户不存在返回 -1。

3.4.6 AdminMainFrame 类(管理员核心界面类)

作为管理员操作的核心可视化界面,基于 Swing 组件实现交易记录的全生命周期管理,核心功能如下:

Java 与数据库应用开发实践报告

交易记录维护:支持右键菜单触发 “修改交易记录” “删除交易记录” “新增交易记录” 操作,点击后弹出交互窗口,校验输入合法性后调用 `AdminService` 层完成数据库操作,并同步更新表格数据;

多条件查询:集成 “用户 ID 输入框 + 交易类型下拉框 + 日期范围选择器”,支持多维度组合查询,点击 “查询” 按钮后异步调用 `AdminService` 的查询方法,避免阻塞界面线程,查询结果实时渲染至表格。

3.4.7 CustomerMainFrame 类(客户核心界面类)

作为客户登录后的核心操作界面,基于 `Swing` 构建可视化交互窗口,封装存款、取款等业务入口,界面适配背景图且布局规整,核心特性如下:

`CustomerMainFrame(int userId):`

初始化界面(标题、大小、居中),加载背景图,创建 3 行 2 列透明功能按钮面板,接收当前登录客户 ID;

核心功能按钮:转账(打开 `TransferFrame`)、存款(`DepositFrame`)、取款(`WithdrawFrame`)、修改密码(`ChangePwdFrame`)、查询余额(调用服务层)、返回(关闭当前窗口并回到主登录界面);

`checkBalance():`调用 `CustomerService` 获取当前用户余额,格式化后弹窗展示,失败则提示错误。

监听器:

①事件源创建 (按钮对象)

代码中通过 `UIUtils.createStyledButton("按钮文本")` 创建各个功能按钮,这些按钮就是「事件源」

// 示例: 转账按钮 (事件源)

```
JButton transferBtn = UIUtils.createStyledButton("转账");
```

②监听器注册 (绑定事件源与处理逻辑)

通过按钮的 `addActionListener()` 方法,将「监听器逻辑」绑定到事件源上。由于 `ActionListener` 是函数式接口 (仅 `actionPerformed` 一个抽象方法),代码中使用 `Lambda` 表达式 替代传统的「匿名内部类」,简化写法。

Java 与数据库应用开发实践报告

③事件触发（用户操作）

当用户点击按钮时，Swing 的「事件分发线程(EDT)」会自动创建 `ActionEvent` 事件对象（封装事件源、触发时间等信息），并将事件传递给已注册的监听器。

④事件处理（执行监听器逻辑）

EDT 调用监听器的 `actionPerformed()` 方法，执行 Lambda 表达式中的业务逻辑。

4 数据库设计

4.1 E-R 图设计

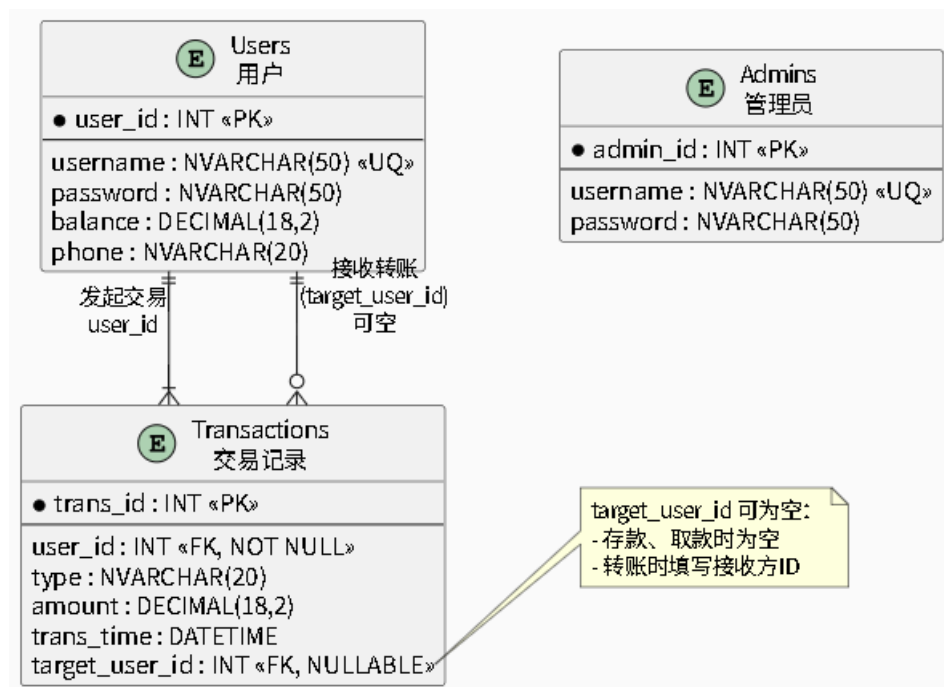


图 4 E-R 图

实体:

用户(Users):`user_id`(PK), `username`(UQ), `password`, `balance`, `phone`

管理员(Admins):`admin_id`(PK), `username`(UQ), `password`

交易记录(Transactions):`trans_id`(PK), `user_id`(FK), `type`, `amount`, `trans_time`, `target_user_id`(FK 可空)

关系:

用户 1:N 交易记录(`user_id`)

Java 与数据库应用开发实践报告

用户 1:1(N) 目标交易记录(target_user_id,转账时使用)

	列名	数据类型	允许 Null 值
▶	user_id	int	☐
	username	nvarchar(50)	☐
	password	nvarchar(50)	☐
	balance	decimal(10, 2)	☒
	phone	varchar(20)	☒
			☐

图 5 客户表数据结构

	列名	数据类型	允许 Null 值
▶	admin_id	int	☐
	username	nvarchar(50)	☐
	password	nvarchar(50)	☐

图 6 管理员表数据结构

	列名	数据类型	允许 Null 值
▶	trans_id	int	☐
	user_id	int	☐
	type	nvarchar(50)	☒
	amount	decimal(10, 2)	☒
	trans_time	datetime	☒
	target_user_id	int	☒

图 7 交易记录表的数据结构

4.2 物理设计与完整建库语句

4.2.1 创建用户表

```
CREATE TABLE dbo.Users (  
    user_id INT IDENTITY(1,1) PRIMARY KEY, -- 客户 ID, 自增主键  
    username NVARCHAR(50) NOT NULL UNIQUE, -- 客户用户名, 唯一且非空  
    password NVARCHAR(50) NOT NULL, -- 客户登录密码, 非空  
    balance DECIMAL(18,2) NOT NULL DEFAULT 0, -- 账户余额, 默认 0  
    phone NVARCHAR(20) -- 客户手机号, 用于身份验证);
```

Java 与数据库应用开发实践报告

4.2.2 创建管理员表

```
CREATE TABLE dbo.Admins (  
    admin_id INT IDENTITY(1,1) PRIMARY KEY, -- 管理员 ID, 自增主键  
    username NVARCHAR(50) NOT NULL UNIQUE, -- 管理用户名, 唯一且非空  
    password NVARCHAR(50) NOT NULL          -- 管理员登录密码, 非空  
);
```

4.2.3 创建交易记录表

```
CREATE TABLE dbo.Transactions (  
    trans_id INT IDENTITY(1,1) PRIMARY KEY, -- 交易记录 ID, 自增主键  
    user_id INT NOT NULL,                    -- 交易发起方用户 ID, 非空  
    type NVARCHAR(20) NOT NULL,              -- 交易类型 (存款/取款/  
转账等), 非空  
    amount DECIMAL(18,2) NOT NULL,          -- 交易金额, 金融精度保  
留 2 位小数  
    trans_time DATETIME NOT NULL DEFAULT GETDATE(), -- 交易时间, 默  
认当前系统时间  
    target_user_id INT NULL,                 -- 交易目标用户 ID (转账场景  
填写, 存款/取款为空)  
    -- 外键约束: 关联发起方用户表  
    CONSTRAINT FK_Transactions_User FOREIGN KEY (user_id)  
REFERENCES dbo.Users(user_id),  
    -- 外键约束: 关联目标方用户表 (允许空值)  
    CONSTRAINT FK_Transactions_TargetUser FOREIGN KEY (target_user_id)  
REFERENCES dbo.Users(user_id)  
);
```


Java 与数据库应用开发实践报告

4.2.4 创建核心视图

```
CREATE VIEW dbo.TransactionView AS
SELECT
    t.trans_id,                -- 交易 ID
    t.user_id AS from_user_id, -- 交易发起方用户 ID
    u1.username AS from_username, -- 交易发起方用户名
    t.target_user_id AS to_user_id, -- 交易目标方用户 ID
    u2.username AS to_username,    -- 交易目标方用户名
    t.type,                      -- 交易类型
    t.amount,                   -- 交易金额
    t.trans_time                -- 交易时间
FROM dbo.Transactions t
JOIN dbo.Users u1 ON t.user_id = u1.user_id        -- 关联发起方用户信息（内连接，确保发起方存在）
LEFT JOIN dbo.Users u2 ON t.target_user_id = u2.user_id; -- 关联目标方用户信息（左连接，兼容存款/取款场景）
```

5 程序设计

5.1 转账事务完整伪代码

```
BEGIN TRANSACTION; -- 开启数据库事务，保障资金操作 ACID 特性
// 步骤 1：验证转出方密码合法性
IF NOT verifyPassword(fromUserId, password) THEN
    ROLLBACK; -- 密码错误，回滚所有操作
    RETURN false;
END IF;
// 步骤 2：根据转入用户名查询目标用户 ID
toUserId = getUserIdByUsername(toUsername);
```

Java 与数据库应用开发实践报告

```
IF toUserId == -1 THEN
    ROLLBACK; -- 转入用户不存在，回滚事务
    RETURN false;
END IF;
// 步骤 3: 校验转出方余额是否充足
IF getBalance(fromUserId) < amount THEN
    ROLLBACK; -- 余额不足，回滚事务
    RETURN "余额不足";
END IF;
// 步骤 4: 转出方账户扣款
UPDATE Users SET balance = balance - amount WHERE user_id =
fromUserId;
// 步骤 5: 转入方账户加款
UPDATE Users SET balance = balance + amount WHERE user_id =
toUserId;
// 步骤 6: 插入转出交易记录（金额记为负数，便于对账）
INSERT INTO Transactions(user_id, type, amount, target_user_id)
VALUES(fromUserId, '转出', -amount, toUserId);
// 步骤 7: 插入转入交易记录
INSERT INTO Transactions(user_id, type, amount, target_user_id)
VALUES(toUserId, '转入', amount, fromUserId);
COMMIT; -- 所有操作执行成功，提交事务
RETURN true;时间复杂度: O(1) 空间复杂度: O(1) 所有资金操作均在 Service
层使用同一个 Connection 完成,确保 ACID。
```

5.2 Swing 线程安全规范

严格遵守 Swing 单线程规则:

// 借助 SwingUtilities.invokeLater 将 UI 操作提交至 EDT（事件调度线程）执

Java 与数据库应用开发实践报告

行 `SwingUtilities.invokeLater() -> { new MainFrame().setVisible(true); }`);

5.3 模块结构与调用关系图

```
Main → MainFrame (系统主窗口)
├─ 客户登录流程
│   ├── 客户登录 → CustomerLoginFrame (客户登录窗口)
│   │   └─ 调用 → CustomerService.login() (客户业务层登录方法)
│   │       └─ 调用 → CustomerDao.login() (客户数据层登录验证)
│   ├── 登录成功 → 进入 CustomerMainFrame (客户核心操作窗口)
│   │   ├── 存款 → DepositFrame (存款窗口) → 调用 CustomerService.deposit() (存款业务逻辑)
│   │   ├── 取款 → WithdrawFrame (取款窗口) → 调用 CustomerService.withdraw() (取款业务逻辑)
│   │   ├── 转账 → TransferFrame (转账窗口) → 调用 CustomerService.transfer() (转账业务逻辑)
│   │   └─ 修改密码 → ChangePwdFrame (修改密码窗口)
│   └─ 登录失败 → 停留在 CustomerLoginFrame 并提示错误
└─ 管理员登录流程
    ├── 管理员登录 → AdminLoginFrame (管理员登录窗口)
    │   └─ 调用 → AdminService.login() (管理员业务层登录方法)
    ├── 登录成功 → 进入 AdminMainFrame (管理员核心操作窗口)
    │   └─ 交易记录管理 (查询/新增/修改/删除) → 调用 AdminDao (管理员数据层) → 关联 TransactionView (交易视图)
    └─ 登录失败 → 停留在 AdminLoginFrame 并提示错误
```

图 8 模块调用图

5.4 核心代码实现

5.4.1 转账事务核心代码

```
public boolean transfer(int fromUserId, String toUsername, double amount, String
password) {
    // 1. 前置业务规则校验
    if (amount <= 0) {
        JOptionPane.showMessageDialog(null, "转账金额必须大于 0!", "错误",
JOptionPane.ERROR_MESSAGE);
        return false; }

    if (!customerDao.verifyPassword(fromUserId, password)) {
        JOptionPane.showMessageDialog(null, "密码错误!", "错误",
JOptionPane.ERROR_MESSAGE);
        return false; }

    // 查询转入方用户 ID
```

Java 与数据库应用开发实践报告

```
int toUserId = customerDao.getUserIdByUsername(toUsername);

if (toUserId == -1) {
    JOptionPane.showMessageDialog(null, "目标用户不存在！", "错误",
JOptionPane.ERROR_MESSAGE);
    return false; }

if (toUserId == fromUserId) {
    JOptionPane.showMessageDialog(null, "不能给自己转账！", "错误",
JOptionPane.ERROR_MESSAGE);
    return false; }

// 2. 事务管控 (try-with-resources 自动关闭连接)
try (Connection conn = BaseDao.getConnection()) {
    conn.setAutoCommit(false); // 关闭自动提交，开启手动事务
    // 调用 DAO 层执行转账数据库操作
    boolean success = customerDao.transfer(fromUserId, toUserId, amount,
conn);

    if (success) {
        conn.commit(); // 事务提交：所有操作永久生效
        JOptionPane.showMessageDialog(null,
            String.format("转账成功！已转%.2f 元给%s", amount,
toUsername),
            "成功", JOptionPane.INFORMATION_MESSAGE);
    } else {
        conn.rollback(); // 事务回滚：撤销所有未提交操作
        JOptionPane.showMessageDialog(null, "转账失败！余额不足或系
统错误", "错误", JOptionPane.ERROR_MESSAGE);
    }
    return success;
}
```

Java 与数据库应用开发实践报告

```
    } catch (SQLException e) {  
        e.printStackTrace(); // 控制台打印异常，便于调试  
        JOptionPane.showMessageDialog(null, "数据库异常,转账失败", "错误",  
JOptionPane.ERROR_MESSAGE);  
        return false; }}
```

5.4.2 管理员可编辑表格核心代码

```
    // 给右键菜单添加「修改」选项  
    popup.add(modifyItem);  
    modifyItem.addActionListener(e -> {  
        // 1. 校验是否选中表格行  
        int selectedRow = transactionTable.getSelectedRow();  
        if (selectedRow == -1) {  
            JOptionPane.showMessageDialog(this, "请先选择一行！");  
            return;  
        }  
        // 2. 获取选中行的交易记录数据  
        int transId = Integer.parseInt((String) tableModel.getValueAt(selectedRow, 0));  
        // 交易 ID（主键）  
        String type = (String) tableModel.getValueAt(selectedRow, 5); // 交易类型  
        String amountStr = (String) tableModel.getValueAt(selectedRow, 6); // 交易  
        金额  
        // 3. 构建修改弹窗组件  
        JComboBox<String> typeCombo = new JComboBox<>(new String[]{"存款", "取款", "转入", "转出"});  
        typeCombo.setSelectedItem(type); // 默认选中当前类型  
        JTextField amountField = new JTextField(amountStr); // 填充当前金额  
        Object[] message = {"交易类型:", typeCombo, "金额(元):", amountField}
```

Java 与数据库应用开发实践报告

```
// 4. 显示修改确认弹窗

int option = JOptionPane.showConfirmDialog(this, message, "修改交易记录",
JOptionPane.OK_CANCEL_OPTION);

if (option == JOptionPane.OK_OPTION) {
// 5. 解析新输入的修改内容

String newType = (String) typeCombo.getSelectedItem();

double newAmount = Double.parseDouble(amountField.getText());

// 6. 调用 Service 层执行修改操作

boolean success = adminService.updateTransaction(transId, newType,
newAmount, null);

if (success) {
// 7. 修改成功：更新表格数据（无需刷新全表）

tableModel.setValueAt(newType, selectedRow, 5);

tableModel.setValueAt(String.format("%.2f", newAmount),
selectedRow, 6);

JOptionPane.showMessageDialog(this, "修改成功！");

}

}

});
```

6 实现结果与测试

6.1 主要界面

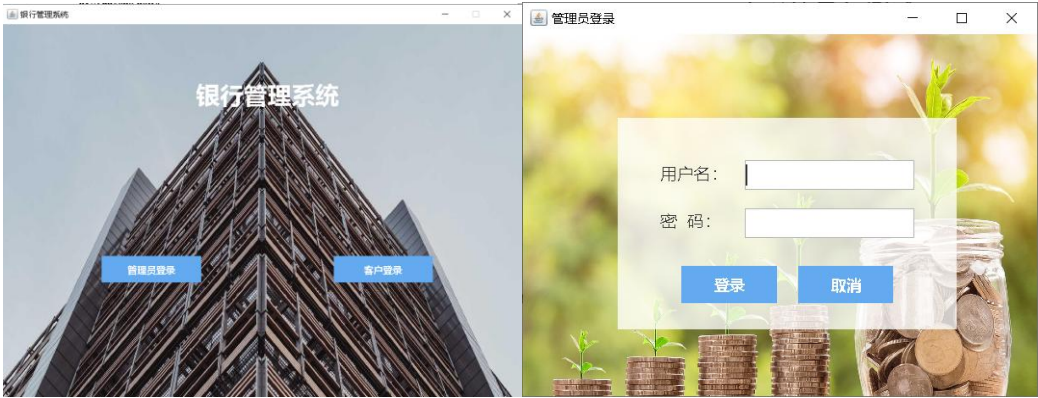


图 9 登录主界面

10 管理员登录主界面

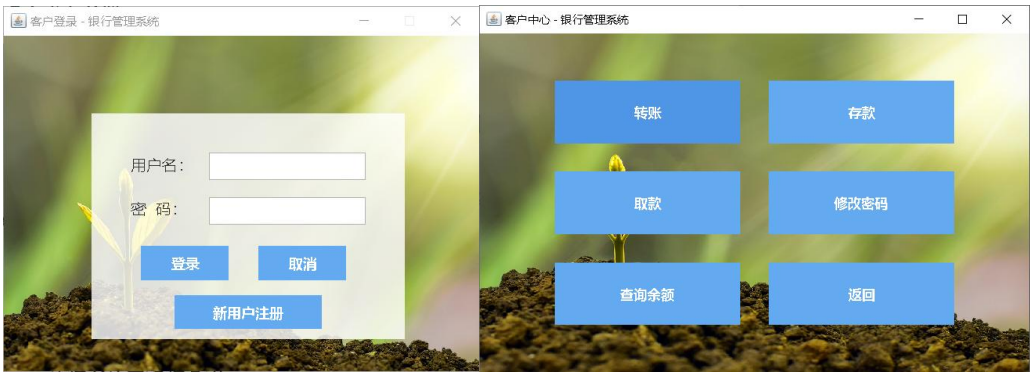


图 11 客户登录主界面

图 12 客户主界面

管理员 - 交易记录管理							
用户ID:		交易类型:	交易日期: (YYYY-MM-DD)		查询		
交易ID	转出方ID	转出方用户名	转入方ID	转入方用户名	类型	金额	时间
78	6	wlc			存款	250.00	2025-11-23 08:53.3
76	12	wcx			转账 (转出)	500.00	2025-11-22 15:10.3
75	12	wcx			存款	1000.00	2025-11-22 15:10.1
74	5	syb	6	wlc	转账	100.00	2025-11-21 20:41.2
71	12	wcx			存款	999999.00	2025-11-21 20:40.1
69	5	syb	6	wlc	转账	1000.00	2025-11-16 18:55.0
68	6	wlc			取款	5628.00	2025-11-16 18:53.5
65	6	wlc			转账 (转出)	250.00	2025-11-16 18:53.3
66	5	syb			转账 (转入)	250.00	2025-11-16 18:53.3
64	5	syb	6	wlc	转账	52200.00	2025-11-16 18:52.1
63	11	zzy			取款	52200.00	2025-11-15 20:11.0
62	6	wlc			存款	1224.00	2025-11-15 20:10.0
59	6	wlc			转账 (转出)	1000.00	2025-11-15 20:09.4
58	6	wlc	5	syb	转账	1000.00	2025-11-15 20:09.1
56	10	zldw			存款	100000.00	2025-11-14 08:46.3
55	6	wlc	5	syb	转账	1200.00	2025-11-14 08:44.5
52	5	syb	6	wlc	转账	1000.00	2025-11-11 21:03.4
48	8	zhl			取款	56210.00	2025-11-09 20:49.2
46	8	zhl			转账 (转出)	500.00	2025-11-09 20:49.1
47	6	wlc			转账 (转入)	500.00	2025-11-09 20:49.1
44	5	syb	6	wlc	转账	6000.00	2025-11-09 20:47.5
43	5	syb	6	wlc	转账	1000.00	2025-11-09 20:39.4
42	7	yfl	6	syb	转账	100.00	2025-11-09 20:23.0
41	6	wlc	4	赵六	转账	10.00	2025-11-09 20:22.3
40	6	wlc			取款	1000.00	2025-11-09 20:22.1
39	6	wlc	6	wlc	转账	100.00	2025-11-09 20:20.0
38	5	syb	6	wlc	转账	100.00	2025-11-09 20:19.3
36	7	yfl			存款	100.00	2025-11-09 18:57.2
33	7	yfl			存款	10000.00	2025-11-09 18:57.0

图 13 管理员主界面

6.2 客户主要功能实现

6.2.1 存款/取款实现

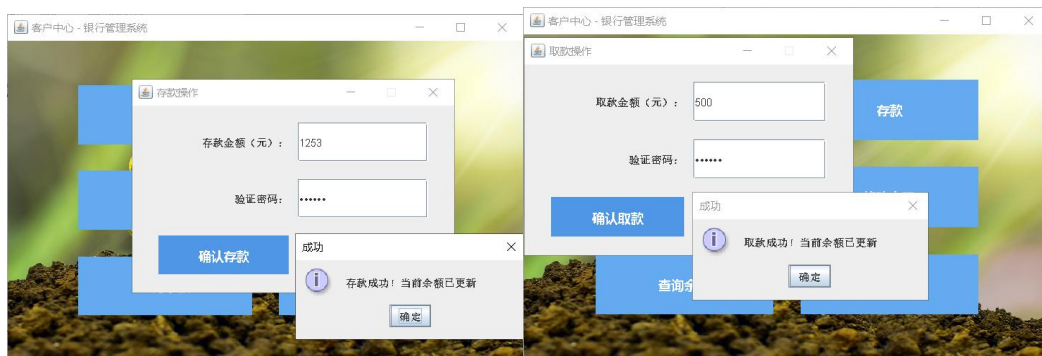


图 14

图 15

6.2.2 转账/查询余额实现



图 16

图 17

6.2.3 修改密码实现



图 18

6.3 管理员主要功能实现

6.3.1 新增交易记录

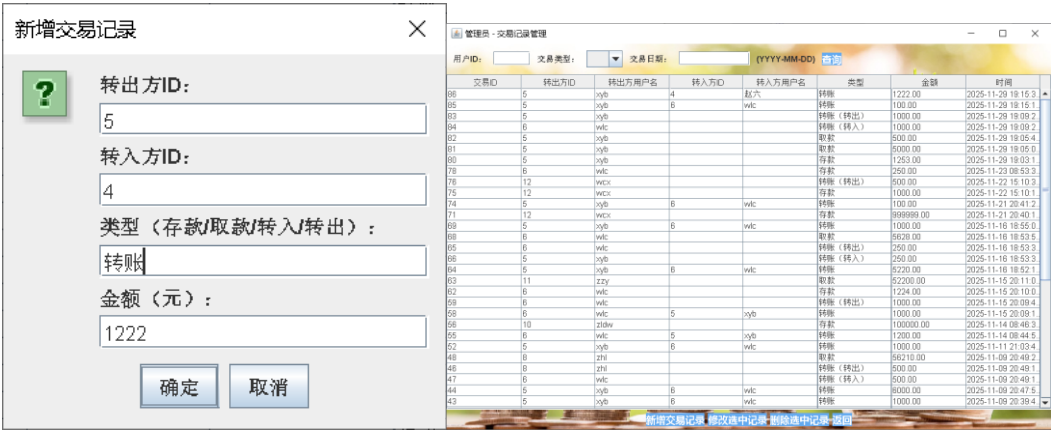


图 19

图 20

Java 与数据库应用开发实践报告

6.3.2 删除选中记录

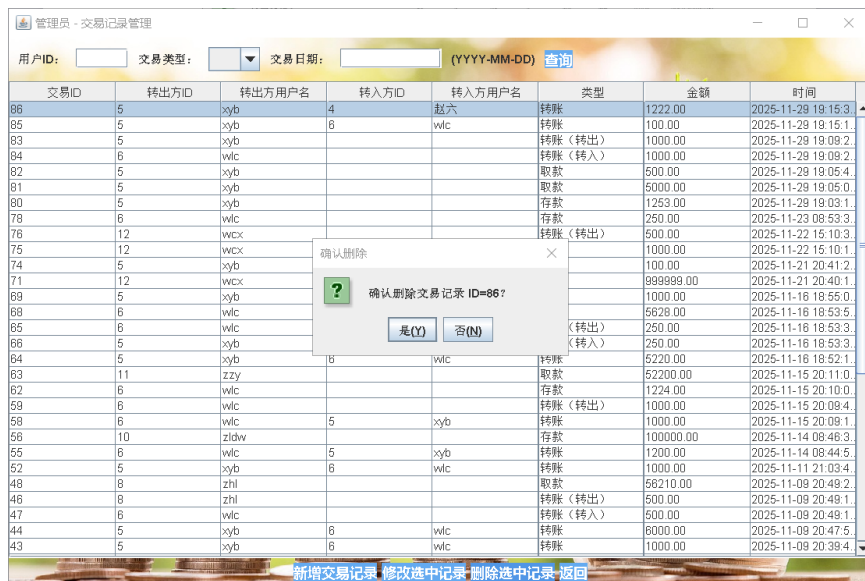


图 21



图 22

Java 与数据库应用开发实践报告

6.3.3 修改选中记录

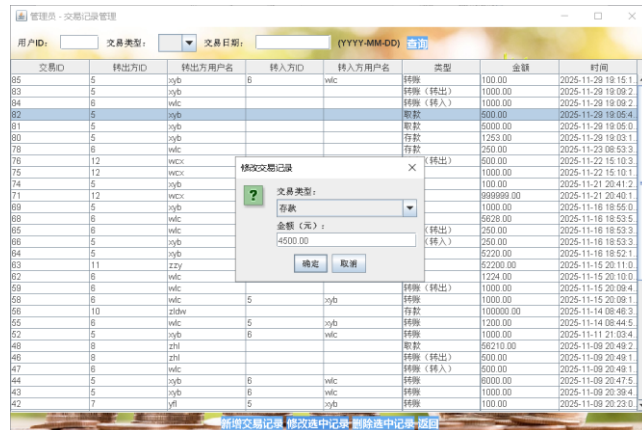


图 23

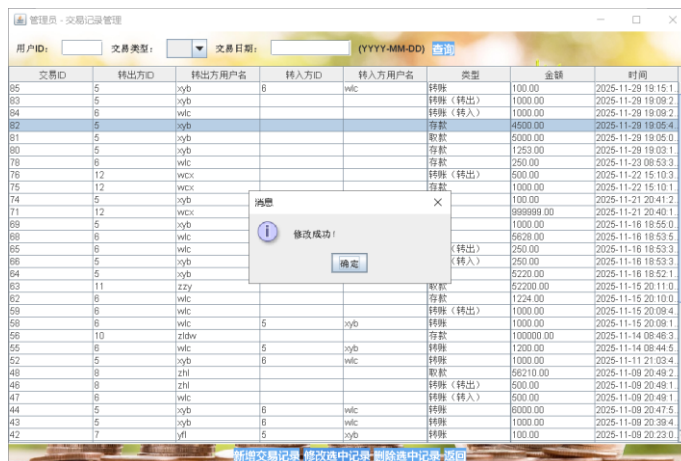


图 24

6.3.4 查询实现

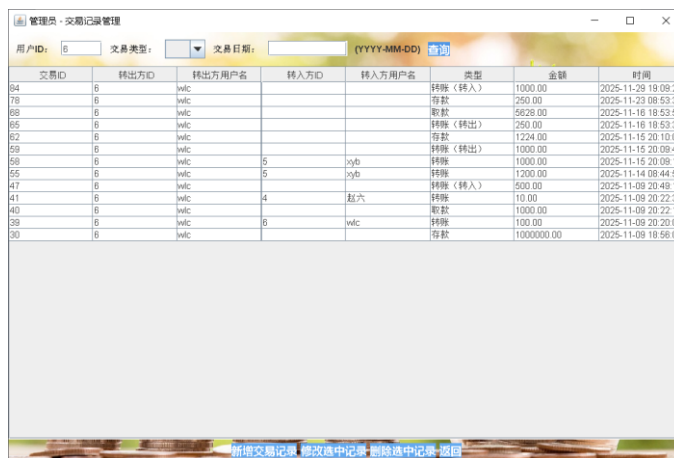


图 25 查询 id=6 用户的记录

7 总结

7.1 课程设计全过程总结

本次课程设计以银行个人业务管理系统为应用背景，旨在全面运用 Java 语言、图形用户界面 (Swing) 和数据库编程 (JDBC) 技术，实现一个功能完善的 C/S 架构桌面应用程序，满足个人客户日常金融操作与管理员交易记录管理的核心需求。

系统设计阶段，采用经典的三层架构（表现层/UI 层、业务逻辑层/Service 层、数据访问层/DAO 层）：表现层基于 Swing 构建可视化交互界面，业务逻辑层封装资金操作的事务管控与业务规则校验，数据访问层设计 BaseDao 基类统一封装 JDBC 连接、资源关闭等通用操作，大幅提升代码复用性与可维护性。

在程序设计与实现阶段，重点攻克两大核心问题：一是基于 Swing 的 GUI 界面布局与事件监听设计，优化按钮、弹窗等交互逻辑，确保客户/管理员操作流程符合金融业务使用习惯；二是资金操作的原子性与一致性保障，针对存款、取款、转账等核心业务，通过 JDBC 手动事务控制（`conn.setAutoCommit(false)`、`conn.commit()/rollback()`），实现“余额变动+交易记录生成”的同步执行，彻底避免资金账实不符。所有数据库操作均采用 PreparedStatement 预处理 SQL 语句，兼顾安全性与性能。

最终，系统成功实现了客户注册登录、资金存取转、密码修改，以及管理员交易记录多条件查询、增删改等核心功能，严格遵循事务一致性与数据安全要求，达到课程设计的预期目标。

7.2 设计成果的优点和不足

优点：

1. 架构清晰，分层解耦：严格遵循“UI 层-Service 层-DAO 层”三层架构，各层职责明确——UI 层仅负责界面交互，Service 层管控业务规则与事务，DAO 层专注数据库操作；DAO 层统一继承 BaseDao，实现 JDBC 代码的高度复用，降低后续维护成本。

Java 与数据库应用开发实践报告

2. 资金事务一致性保障：针对存款、取款、转账等核心资金操作，通过显式的 JDBC 事务控制，确保“余额变动+交易记录插入”等关键步骤同时成功或回滚，彻底规避金融场景下资金账实不符的风险。

3. 安全性与稳定性突出：数据库操作全部使用 `PreparedStatement` 实现，有效防范 SQL 注入攻击；敏感操作（转账、改密）增加密码/手机号二次验证，`BaseDao` 中通过 `try-with-resources` 语法自动关闭 `Connection`、`PreparedStatement` 等资源，避免 JDBC 资源泄漏。

不足：

1. 界面交互体验局限：系统基于 Java Swing 构建 GUI，界面样式偏传统，视觉效果与交互体验远不及现代 Web/移动端金融应用，缺乏个性化定制与可视化数据展示。

2. 并发处理能力不足：作为单机桌面应用，所有业务逻辑均在 AWT 事件分发线程（EDT）中执行；若数据库操作（如大批量交易记录查询）耗时较长，易导致界面卡顿、无响应，未针对多用户并发场景设计多线程异步处理机制。

7.3 个人学习心得

通过本次银行管理系统的课程设计，我对 Java 桌面应用开发的全流程有了系统且深入的理解，同时对金融场景下的编程规范与数据安全有了全新认知。

首先，巩固并深化了面向对象编程思想：学会将银行业务中的核心实体（客户、交易记录、管理员）抽象为 Java 类，通过封装（如 `CustomerService` 封装转账事务）、继承（如 DAO 层继承 `BaseDao`）实现代码结构化，理解了“高内聚、低耦合”在实际项目中的落地方式。

其次，熟练掌握了 JDBC 数据库编程的核心要点：不仅能灵活运用 `Connection`、`PreparedStatement`、`ResultSet` 等接口完成 CRUD 操作，更深刻理解了金融场景下事务控制的必要性——转账业务中“一方扣款、一方加款”的原子性要求，让我真正掌握了 `conn.setAutoCommit(false)`、`commit()`、`rollback()` 的实际应用，而非仅停留在理论层面。

最后，发现了自身的不足与改进方向：在处理资金一致性问题时，我意识到

Java 与数据库应用开发实践报告

对数据库并发控制、分布式事务等高级知识的欠缺；界面设计中也暴露了交互体验设计思维的不足。让我对程序设计的严谨性和实用性有了更深刻的理解。

附录：源程序

```
/*
*****

* 文件名称:   BaseDao.java

// 数据库连接工具类: 统一封装连接逻辑, 供所有 Dao 类调用, 简化维护

public class BaseDao {

    // 格式: jdbc:sqlserver://[地址]:[端口];databaseName=[库
    名];trustServerCertificate=true (跳过 SSL 验证)

    private static final String URL =
    "jdbc:sqlserver://localhost:1433;databaseName=BankDB;trustServerCertificate=true;";

    private static final String USER = "sa";

    private static final String PASSWORD = "123456";

    // 静态方法: 获取数据库连接

    // @return 数据库连接对象

    // @throws SQLException 连接失败时抛出 (如配置错误、数据库未启动),
    由调用者处理

    public static Connection getConnection() throws SQLException {

        // 自动加载驱动, 返回连接对象

        return DriverManager.getConnection(URL, USER, PASSWORD);

    }

}
*/
```

```
/*
*****
```

```
* 文件名称:   CustomerDao.java

public class CustomerDao {

    // 1.客户登录验证

    // @param username 用户名

    // @param password 密码

    // @return 验证通过返回用户 ID, 失败返回-1
```

Java 与数据库应用开发实践报告

```
public int login(String username, String password) {  
    String sql = "SELECT user_id FROM dbo.Users WHERE username = ?  
AND password = ?";  
  
    try (Connection conn = BaseDao.getConnection()); // 获取数据库连接  
        PreparedStatement ps = conn.prepareStatement(sql)) { // 预编译  
SQL 语句  
  
        ps.setString(1, username);  
        ps.setString(2, password);  
  
        ResultSet rs = ps.executeQuery(); // 执行查询，返回结果集  
        return rs.next() ? rs.getInt("user_id") : -1;  
        // 若结果集有数据，返回用户 ID；否则返回-1  
    } catch (SQLException e) {  
        e.printStackTrace();  
        return -1;  
    }  
}  
  
// 2.校验客户密码（用于敏感操作验证）  
// @param userId 客户 ID  
// @param password 待校验密码  
// @return 校验通过返回 true，失败返回 false  
public boolean verifyPassword(int userId, String password) {  
    String sql = "SELECT user_id FROM dbo.Users WHERE user_id = ?  
AND password = ?";  
  
    try (Connection conn = BaseDao.getConnection();  
        PreparedStatement ps = conn.prepareStatement(sql)) {  
        ps.setInt(1, userId);  
        ps.setString(2, password);
```


Java 与数据库应用开发实践报告

```
        return ps.executeQuery().next();
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

// 3.存款操作（事务连接，更新余额+插入交易记录）

// @param userId 存款客户 ID
// @param amount 存款金额
// @param conn 事务连接（确保操作原子性）
// @return 操作成功返回 true，失败返回 false
// @throws SQLException SQL 执行异常

public boolean deposit(int userId, double amount, Connection conn) throws
SQLException {
    // 更新用户余额

    String updateSql = "UPDATE dbo.Users SET balance = balance + ?
WHERE user_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
        ps.setDouble(1, amount);
        ps.setInt(2, userId);
        if (ps.executeUpdate() <= 0) return false;
    }

    // 插入存款交易记录

    String insertSql = "INSERT INTO dbo.Transactions (user_id, [type],
amount, trans_time) VALUES (?, ?, ?, GETDATE())";

    try (PreparedStatement ps = conn.prepareStatement(insertSql)) {
        ps.setInt(1, userId);
```

Java 与数据库应用开发实践报告

```
        ps.setString(2, "存款");
        ps.setDouble(3, amount);
        return ps.executeUpdate() > 0;
    }
}

// 4.取款操作（事务连接，校验余额+更新余额+插入交易记录）
// @param userId 取款客户 ID
// @param amount 取款金额
// @param conn 事务连接（确保操作原子性）
// @return 操作成功返回 true，失败返回 false
// @throws SQLException SQL 执行异常

public boolean withdraw(int userId, double amount, Connection conn) throws
SQLException {
    // 校验余额充足
    String checkSql = "SELECT balance FROM dbo.Users WHERE user_id
= ? AND balance >= ?";

    try (PreparedStatement ps = conn.prepareStatement(checkSql)) {
        ps.setInt(1, userId);
        ps.setDouble(2, amount);
        if (!ps.executeQuery().next()) return false;
    }

    // 更新用户余额
    String updateSql = "UPDATE dbo.Users SET balance = balance - ?
WHERE user_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
        ps.setDouble(1, amount);
        ps.setInt(2, userId);
```

Java 与数据库应用开发实践报告

```
        if (ps.executeUpdate() <= 0) return false;
    }

    // 插入取款交易记录

    String insertSql = "INSERT INTO dbo.Transactions (user_id, [type],
amount, trans_time) VALUES (?, ?, ?, GETDATE())";

    try (PreparedStatement ps = conn.prepareStatement(insertSql)) {
        ps.setInt(1, userId);
        ps.setString(2, "取款");
        ps.setDouble(3, amount);
        return ps.executeUpdate() > 0;
    }
}

// 5.转账操作（事务连接，校验余额+更新双方余额+插入两条交易记录）
// @param fromUserId 转出客户 ID
// @param toUserId 转入客户 ID
// @param amount 转账金额
// @param conn 事务连接（确保操作原子性）
// @return 操作成功返回 true，失败返回 false
// @throws SQLException SQL 执行异常

public boolean transfer(int fromUserId, int toUserId, double amount,
Connection conn) throws SQLException {
    // 校验转出用户余额充足

    String checkFromSql = "SELECT balance FROM dbo.Users WHERE
user_id = ? AND balance >= ?";

    try (PreparedStatement ps = conn.prepareStatement(checkFromSql)) {
        ps.setInt(1, fromUserId);
        ps.setDouble(2, amount);
```

Java 与数据库应用开发实践报告

```
        if (!ps.executeQuery().next()) return false;
    }
    // 校验转入用户存在
    String checkToSql = "SELECT user_id FROM dbo.Users WHERE
user_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(checkToSql)) {
        ps.setInt(1, toUserId);
        if (!ps.executeQuery().next()) return false;
    }
    // 转出用户扣钱
    String updateFromSql = "UPDATE dbo.Users SET balance = balance - ?
WHERE user_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(updateFromSql)) {
        ps.setDouble(1, amount);
        ps.setInt(2, fromUserId);
        if (ps.executeUpdate() <= 0) return false;
    }
    // 转入用户加钱
    String updateToSql = "UPDATE dbo.Users SET balance = balance + ?
WHERE user_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(updateToSql)) {
        ps.setDouble(1, amount);
        ps.setInt(2, toUserId);
        if (ps.executeUpdate() <= 0) return false;
    }
    // 插入转出交易记录
    String insertFromSql = "INSERT INTO dbo.Transactions (user_id, [type],
```

Java 与数据库应用开发实践报告

```
amount, trans_time) VALUES (?, ?, ?, GETDATE());

    try (PreparedStatement ps = conn.prepareStatement(insertFromSql)) {
        ps.setInt(1, fromUserId);
        ps.setString(2, "转账（转出）");
        ps.setDouble(3, amount);
        ps.executeUpdate();
    }

    // 插入转入交易记录

    String insertToSql = "INSERT INTO dbo.Transactions (user_id, [type],
amount, trans_time) VALUES (?, ?, ?, GETDATE());

    try (PreparedStatement ps = conn.prepareStatement(insertToSql)) {
        ps.setInt(1, toUserId);
        ps.setString(2, "转账（转入）");
        ps.setDouble(3, amount);
        return ps.executeUpdate() > 0;
    }
}

// 6.验证客户手机号（修改密码前校验）
// @param userId 客户 ID
// @param phone 待校验手机号
// @return 校验通过返回 true，失败返回 false

public boolean verifyPhone(int userId, String phone) {

    String sql = "SELECT user_id FROM dbo.Users WHERE user_id = ?
AND phone = ?";

    try (Connection conn = BaseDao.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, userId);
```

Java 与数据库应用开发实践报告

```
        ps.setString(2, phone);

        return ps.executeQuery().next();
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

// 7.修改客户密码

// @param userId 客户 ID
// @param newPassword 新密码
// @return 修改成功返回 true，失败返回 false

public boolean updatePassword(int userId, String newPassword) {
    String sql = "UPDATE dbo.Users SET password = ? WHERE user_id
= ?";

    try (Connection conn = BaseDao.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, newPassword);
        ps.setInt(2, userId);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

// 8.通过用户名查询用户 ID（转账时使用）

// @param username 用户名
// @return 存在返回用户 ID，不存在返回-1
```

Java 与数据库应用开发实践报告

```
public int getUserIdByUsername(String username) {  
    String sql = "SELECT user_id FROM dbo.Users WHERE username = ?";  
    try (Connection conn = BaseDao.getConnection();  
        PreparedStatement ps = conn.prepareStatement(sql)) {  
        ps.setString(1, username);  
        ResultSet rs = ps.executeQuery();  
        return rs.next() ? rs.getInt("user_id") : -1;  
    } catch (SQLException e) {  
        e.printStackTrace();  
        return -1;  
    }  
}
```

// 9.根据客户 ID 查询余额

// @param userId 客户 ID

// @return 成功返回余额，用户不存在/异常返回-1

```
public double getBalance(int userId) {  
    String sql = "SELECT balance FROM dbo.Users WHERE user_id = ?";  
    try (Connection conn = BaseDao.getConnection();  
        PreparedStatement ps = conn.prepareStatement(sql)) {  
        ps.setInt(1, userId);  
        ResultSet rs = ps.executeQuery();  
        return rs.next() ? rs.getDouble("balance") : -1;  
    } catch (SQLException e) {  
        e.printStackTrace();  
        return -1;  
    }  
}
```

Java 与数据库应用开发实践报告

Java 与数据库应用开发实践成绩评价表

课程目标	评价依据	评价指标及标准	满分	学生自评	教师确认
课程目标 1:掌握软件工程全周期的基本方法和技术,进行问题分析、系统建模,了解影响软件设计目标和技术方案的各种因素。	课程设计报告中“1 项目描述”和“2 系统分析”部分为主,涵盖各部分。	项目描述清晰、完整。功能完善,工作量充实。报告按模板要求写作,(手写)字迹清楚,书写工整,或(打印)版面在边距、行距等方面处理恰当,整体可读性好。	20	18	
课程目标 2:能够针对特定需求,进行程序设计,完成软件模块的设计与实现。	课程设计报告中“5 程序设计与调试”部分和“6 程序实现结果及测试”部分	对系统中关键的数据处理设计处理算法。程序模块功能划分合理,接口清晰、高效。解决问题运用较复杂的 Java 技术(多线程、网络、GUI 等)。	30	27	
课程目标 3:能够进行软件过程的设计或软件系统的设计与实现,完成具体系统的设计,在设计中体现创新意识。	课程设计报告中“3 系统设计”部分和“4 数据库设计”部分	确定类主要的成员以及接口的成员。对引入的每个成员要详细说明其功用、初值和操作的特点。进行数据库设计,运用视图、索引、触发器、存储过程、函数、游标等机制。	20	17	
课程目标 4:在软件设计中,特别是在选题时,能够考虑安全、健康、法律、文化及环境等制约因素。	整个设计过程中	答辩:安全、健康、法律、文化及环境等制约因素	15	13	
课程目标 5:分组协作,能够在团队中独立或合作开展工作,具有团队合作意识和良好的沟通能力。	整个设计过程中	答辩:团队合作意识	15	12	
合 计			100	87	